

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5

7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Judy Allen J

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Answer:

For an e-commerce system containing 10 million product records, a Hashed File Organization is suitable when products are mainly retrieved using a unique key such as ProductID.

In hashed file organization, a hash function is applied to the ProductID to determine the block where the record is stored.

Example:

where M represents the number of available buckets.

Justification

1. Product records can be retrieved directly using ProductID.
2. Search time is approximately $O(1)$ on average.
3. Insertions are also fast because the hash function directly identifies the appropriate bucket.
4. It avoids scanning millions of records.
5. Collision handling methods such as chaining or overflow buckets can be used.

Cost Analysis

Assume:

- Number of records = 10,000,000
- Record size = 100 bytes
- Block size = 4096 bytes

Records per block:

Number of blocks:

With a heap file, searching for one product may require approximately:
block accesses on average.

With hashing, the required product can normally be found in approximately 1–2 block accesses.

Conclusion

Therefore, hashed file organization is recommended for the e-commerce product database because it provides very fast exact-match retrieval and insertion with considerably lower disk I/O cost.

Answer:

The index can be created as:

Reason for Selection

TransactionDate is placed second because the system frequently retrieves transactions within a particular date range.

SELECT *

WHERE AccountNumber = 10001

Advantages

- ## Conclusion

MySQL Output:

```
mysql>
mysql> CREATE INDEX idx_account_date
-> ON Transactions(AccountNumber, TransactionDate);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> SHOW INDEX FROM Transactions;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non-unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | P |
+-----+-----+-----+-----+-----+-----+-----+-----+
| transactions | NULL | Index_type | Comment | Index_comment | Visible | Expression |  |  |
+-----+-----+-----+-----+-----+-----+-----+-----+
| transactions | 0 | PRIMARY | 1 | TransactionID | A | 0 | NULL |  |
| NULL | YES | BTREE |  | YES | NULL |  |  |  |
| transactions | 1 | idx_account_date | 1 | AccountNumber | A | 0 | NULL |  |
| NULL | YES | BTREE |  | YES | NULL |  |  |  |
| transactions | 1 | idx_account_date | 2 | TransactionDate | A | 0 | NULL |  |
| NULL | YES | BTREE |  | YES | NULL |  |  |  |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.03 sec)
```

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Answer:

A healthcare database should use a multi-level indexing approach because patient records are searched using both PatientID and DoctorName.

Proposed Index Structure

1. Primary Index on PatientID

Since PatientID is unique, a primary B+ Tree index can be created.

CREATE INDEX idx_patient

ON Patient(PatientID);

This enables fast retrieval of an individual patient.

2. Secondary Index on DoctorName

A secondary index can be created on DoctorName.

CREATE INDEX idx_doctor

ON Patient(DoctorName);

Since one doctor may treat several patients, the secondary index may point to multiple patient records.

Multi-Level Operation

The upper level of the index stores search-key values and pointers to lower-level index blocks.

The lower-level index then points toward the actual patient record.

Thus:

Index Level 1 → Index Level 2 → Data Block → Patient Record

Advantages

- Fast PatientID search
- Fast doctor-wise patient search
- Reduced disk accesses
- Efficient handling of large healthcare datasets
- B+ Tree indexes remain balanced as records grow

Conclusion

Therefore, using a primary B+ Tree index on PatientID and secondary B+ Tree index on DoctorName provides an efficient multi-level indexing solution.

MySQL Output:

```
mysql> CREATE INDEX idx_doctor
-> ON Patient(DoctorName);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql>
```

```
mysql> SHOW INDEX FROM Patient;
```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+
| patient | 0 | PRIMARY | 1 | PatientID | A | 0 | NULL | NULL |
BTREE | | | YES | NULL |
| patient | 1 | idx_doctor | 1 | DoctorName | A | 0 | NULL | NULL | YES
BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer:

For a logistics company where shipment records are continuously inserted, deleted and searched, a B+ Tree is more suitable than a B-Tree.

Comparison

Feature	B-Tree	B+ Tree
Data storage	Internal and leaf nodes	Only leaf nodes
Search	Efficient	More efficient and uniform
Range search	Moderate	Excellent
Sequential access	Less efficient	Very efficient
Insert/Delete	Efficient	Efficient
Leaf nodes	Usually not linked	Linked sequentially

In a B+ Tree, internal nodes contain only search keys and pointers. Actual shipment records or record pointers are stored in leaf nodes.

Because internal nodes contain fewer data values, more keys can fit into one disk block.

Therefore, the height of the tree is reduced.

Insertions and deletions are handled using:

- Node splitting
- Node merging
- Redistribution of keys

The tree remains balanced after these operations.

Conclusion

The B+ Tree is preferred because it supports frequent insertions and deletions while maintaining balanced structure and also provides efficient exact-match, sequential and range searches required in logistics applications.

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing.

Answer:

For an HR system containing 5000 employees, EmployeeID can be used as the hash key.

A simple hash function can be:

This creates 1000 initial buckets.

For example:

Therefore, EmployeeID 2457 will be stored in bucket 457.

Static Hashing

In static hashing, the number of buckets remains fixed.

Advantages:

- Simple implementation
- Fast exact-match search
- Low overhead

Disadvantages:

- Performance decreases as employee records increase.
- Overflow buckets may be required.
- Large numbers of collisions may occur.

Extendible Hashing

Extendible hashing dynamically increases or decreases the number of buckets.

It uses a directory containing pointers to buckets.

When a bucket becomes full:

1. The bucket is split.
2. Local depth is increased.
3. Directory size may be increased.
4. Records are redistributed.

Comparison

Feature	Static Hashing	Extendible Hashing
Number of buckets	Fixed	Dynamic
Growth handling	Poor	Excellent
Overflow	More likely	Less likely
Storage overhead	Low	Moderate
Search speed	Fast initially	Consistently fast

Conclusion

For exactly 5000 employees with little growth, static hashing is sufficient. However, for a growing organization, extendible hashing is preferred because it dynamically handles increasing employee records while minimizing overflow and collisions.

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Answer:

The airline reservation system requires two different types of searches:

- Exact search using FlightNumber
- Range search using FlightDate

Therefore, a combined indexing strategy should be used.

1. Hash/Unique Index on FlightNumber

FlightNumber can be indexed for fast point queries.

Example:

```
CREATE UNIQUE INDEX idx_flight_no  
ON Flight(FlightNumber);
```

This enables queries such as:

```
SELECT *  
FROM Flight  
WHERE FlightNumber = 'AI101';
```

2. B+ Tree Index on FlightDate

A B+ Tree index should be created on FlightDate because B+ Trees efficiently support range queries.

```
CREATE INDEX idx_flight_date  
ON Flight(FlightDate);
```

Example:

```
SELECT *  
FROM Flight  
WHERE FlightDate BETWEEN '2026-09-01' AND '2026-09-10';
```

Advantages

- FlightNumber search provides fast exact retrieval.
- B+ Tree supports ordered date retrieval.
- Date ranges can be processed without scanning the entire table.
- Each index is optimized for its corresponding query type.

Conclusion

The airline reservation database should combine an index on FlightNumber with a B+ Tree index on FlightDate to efficiently support both point and range queries.

MySQL Output:

```
mysql> CREATE INDEX idx_flight_date  
-> ON Flight(FlightDate);  
Query OK, 0 rows affected (0.05 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```



```
mysql>  
mysql> SHOW INDEX FROM Flight;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
Index_type	Comment	Index_comment	Visible	Expression					
flight	0	PRIMARY	1	FlightNumber	A	0	NULL	NULL	YES
BTREE			YES	NULL					
flight	1	idx_flight_date	1	FlightDate	A	0	NULL	NULL	YES
S BTREE			YES	NULL					

```
2 rows in set (0.01 sec)
```

Answer:

A composite index can be created as:

Example query:

Working

Within the selected department, CGPA values are maintained in sorted order, allowing efficient range searching.

Benefits

- ## Conclusion

A heap file organization with a composite B+ Tree secondary index on (Department, CGPA) provides a good balance between insertion performance and fast search operas:

MySQL Output:

```
mysql> CREATE INDEX idx_dept_cgpa  
-> ON Student(Department, CGPA);  
Query OK, 0 rows affected (0.04 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql>  
mysql> SHOW INDEX FROM Student;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
Index_type Comment Index_comment Visible Expression									
student	0	PRIMARY		1 StudentID	A	0	NULL	NULL	
BTREE			YES	NULL					
student	1	idx_dept_cgpa		1 Department	A	0	NULL	NULL	YES
BTREE			YES	NULL					
student	1	idx_dept_cgpa		2 CGPA	A	0	NULL	NULL	YES
BTREE			YES	NULL					

```
3 rows in set (0.01 sec)
```

Q8. Analyze disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Answer:

Given:

Number of records:

Record size:

Block size:

Blocking Factor

Number of Blocks

Heap File

Heap files store records without ordering.

Approximate blocks required:

Insertion is very fast, but search may require scanning approximately half of the blocks.

Average search:

Sorted File

A sorted file also requires approximately:

However, binary search can be performed.

Search cost:

Therefore, approximately 11 block accesses are required.

Insertion is more expensive because records must maintain sorted order.

Hashed File

Hashing distributes records into buckets.

Exact-match searching usually requires approximately:

However, additional space may be needed for empty buckets and collision overflow areas.

Comparison

Organization	Approx. Blocks	Search Performance	Insertion
Heap	1235	~618 average block accesses	Very fast
Sorted	1235	~11 block accesses	Costly
Hashed	≥ 1235	~1–2 average accesses	Fast

Conclusion

Heap files provide efficient insertion, sorted files provide good range and ordered searching, while hashed files provide the best exact-match search performance at the expense of some additional bucket space.

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Answer:

A social media platform containing 500 million posts requires multiple indexes because users may search posts by user, posting time and hashtag.

1. Composite Index on UserID and Timestamp

CREATE INDEX idx_user_time

ON Posts(UserID, PostTimestamp);

This supports queries such as:

SELECT *

FROM Posts

WHERE UserID = 1001

ORDER BY PostTimestamp DESC;

It also supports retrieving a user's posts within a time range.

2. Index on Hashtag

A separate index should be maintained for hashtags.

CREATE INDEX idx_hashtag

ON Posts(Hashtag);

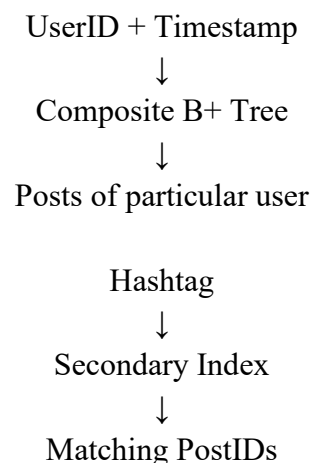
For a production social-media system where one post can contain many hashtags, hashtags should preferably be normalized into a separate table such as:

POST_HASHTAG(PostID, Hashtag)

with an index on:

(Hashtag, PostID)

Proposed Structure



Advantages

- Efficient user timeline generation
- Fast chronological retrieval
- Fast hashtag searches
- Reduced scanning of 500 million records
- Scalable indexing structure

Conclusion

A composite B+ Tree index on (UserID, Timestamp) together with a secondary hashtag index provides an efficient and scalable solution for indexing hundreds of millions of social-media posts.

MySQL Output:

```
mysql> CREATE INDEX idx_hashtag  
-> ON Posts(Hashtag);  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```



```
mysql>  
mysql> SHOW INDEX FROM Posts;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
Index_type	Comment	Index_comment	Visible	Expression					
posts BTREE	0	PRIMARY	YES	1 PostID NULL	A	0		NONE	NONE
posts BTREE	1	idx_user_time	YES	1 UserID NULL	A	0		NONE	NONE
posts BTREE	1	idx_user_time	YES	2 PostTimestamp NULL	A	0		NONE	NONE
posts BTREE	1	idx_hashtag	YES	1 Hashtag NULL	A	0		NONE	NONE

```
4 rows in set (0.01 sec)
```

Q10. Compare the performance of Heap File, Sorted File and Hashed File for a supermarket billing system with 1 million daily transactions.

Answer:

A supermarket billing system processing 1 million transactions per day requires frequent insertions and fast retrieval of individual transaction records.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	Very fast	Slow	Fast
Delete	Moderate	Slow	Fast for key-based deletion
Exact Search	Slow	Fast	Very fast
Range Search	Slow	Very fast	Poor
Maintenance	Low	High	Moderate

1. Heap File

Records are stored in the order in which they arrive.

Insert: approximately $O(1)$

New transactions can simply be appended to the file.

Search: approximately $O(n)$

For 1 million transactions, a large number of records may need to be scanned.

Therefore, heap organization is excellent for continuous transaction insertion but poor for searching without indexes.

2. Sorted File

Records are maintained according to a search key.

Search:

Binary search can locate transaction records efficiently.

However, insertion and deletion are costly because the sorted order must be maintained.

Therefore, a sorted file is useful for reports and range-based transaction analysis.

3. Hashed File

A hash function can map TransactionID directly to a bucket.

Average search:

Average insertion:

Hashing therefore provides excellent performance for exact TransactionID searches.

However, hashing is not suitable for date or amount range queries.

Performance Summary

For 1 million daily transactions:

Heap:

Insert → Excellent

Search → Poor

Delete → Moderate

Sorted:

Insert → Poor

Search → Good

Range Search → Excellent

Hashed:

Insert → Excellent

Exact Search → Excellent

Range Search → Poor

Conclusion

For the supermarket billing system, heap organization is suitable for rapidly recording incoming transactions, while hashing is suitable when fast TransactionID retrieval is the main requirement. A practical industry solution would use a heap-like transaction table together with appropriate B+ Tree indexes for reporting and key-based indexes for transaction lookup.